

# Unit Testing Assignment – Calculator (.NET NUnit)

Name: Tushar Dev Singh

## **Calculator Class Code:**

```
using System;

using System.Collections.Generic;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

namespace CalculatorLibrary
{

    public class Calculator
    {

        public double Add(double a, double b)
        {

            return a + b;

        }

        public double Subtract(double a, double b)
        {

            return a - b;

        }

        public double Multiply(double a, double b)
        {

            return a * b;

        }

    }

}
```

```

public double Divide(double a, double b)
{
    if (b == 0)
    {
        throw new DivideByZeroException("Cannot divide by zero.");
    }

    return a / b;
}
}
}

```

The screenshot shows a Visual Studio editor window titled "CalculatorLibrary". The code is in C# and defines a namespace "CalculatorLibrary" containing a class "Calculator". The class has four public methods: "Add", "Subtract", "Multiply", and "Divide". Each method is annotated with test results: "Add" (2 references, 2/2 passing), "Subtract" (1 reference, 1/1 passing), "Multiply" (2 references, 2/2 passing), and "Divide" (2 references, 2/2 passing). The "Divide" method includes a check for division by zero, throwing a "DivideByZeroException" if the divisor is zero. The code is as follows:

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace CalculatorLibrary
8  {
9
10     2 references
11     public class Calculator
12     {
13         2 references 2/2 passing
14         public double Add(double a, double b)
15         {
16             return a + b;
17         }
18
19         1 reference 1/1 passing
20         public double Subtract(double a, double b)
21         {
22             return a - b;
23         }
24
25         2 references 2/2 passing
26         public double Multiply(double a, double b)
27         {
28             return a * b;
29         }
30
31         2 references 2/2 passing
32         public double Divide(double a, double b)
33         {
34             if (b == 0)
35             {
36                 throw new DivideByZeroException("Cannot divide by zero.");
37             }
38             return a / b;
39         }
40     }
41 }

```

Unit Test Code (NUnit):

```
namespace CalculatorLibrary.Test
```

```
{
```

```
    public class CalculatorTests
```

```
    {
```

```
        private Calculator calc;
```

```
        [SetUp]
```

```
        public void Setup()
```

```
        {
```

```
            calc = new Calculator();
```

```
        }
```

```
        // ADD
```

```
        [Test]
```

```
        public void Add_Test()
```

```
        {
```

```
            Assert.AreEqual(5, calc.Add(2, 3));
```

```
        }
```

```
        [Test]
```

```
        public void Add_WithZero()
```

```
        {
```

```
            Assert.AreEqual(10, calc.Add(10, 0));
```

```
        }
```

```
// SUBTRACT
```

```
[Test]
```

```
public void Subtract_Test()
```

```
{
```

```
    Assert.AreEqual(5, calc.Subtract(10, 5));
```

```
}
```

```
// MULTIPLY
```

```
[Test]
```

```
public void Multiply_Test()
```

```
{
```

```
    Assert.AreEqual(20, calc.Multiply(4, 5));
```

```
}
```

```
[Test]
```

```
public void Multiply_ByZero()
```

```
{
```

```
    Assert.AreEqual(0, calc.Multiply(10, 0));
```

```
}
```

```
// DIVIDE
```

```
[Test]
```

```
public void Divide_Test()
```

```
{
```

```
    Assert.AreEqual(5, calc.Divide(10, 2));
```

```
}
```

```
// DIVIDE BY ZERO
```

```
[Test]
```

```
public void Divide_ByZero_Test()
```

```
{
```

```
    Assert.Throws<DivideByZeroException>(() => calc.Divide(10, 0));
```

```
}
```

```
}
```

```
}
```



CalculatorLibrary.Test



Calcu

```
3 public class CalculatorTests
4 {
5     private Calculator calc;
6
7     [Setup]
8     public void Setup()
9     {
10         calc = new Calculator();
11     }
12
13     // ADD
14     [Test]
15     public void Add_Test()
16     {
17         Assert.AreEqual(5, calc.Add(2, 3));
18     }
19
20     [Test]
21     public void Add_WithZero()
22     {
23         Assert.AreEqual(10, calc.Add(10, 0));
24     }
25
26     // SUBTRACT
27     [Test]
28     public void Subtract_Test()
29     {
30         Assert.AreEqual(5, calc.Subtract(10, 5));
31     }
32
33     // MULTIPLY
34     [Test]
35     public void Multiply_Test()
36     {
37         Assert.AreEqual(20, calc.Multiply(4, 5));
38     }
39
40     [Test]
41     public void Multiply_ByZero()
42     {
43         Assert.AreEqual(0, calc.Multiply(10, 0));
44     }
45
46     // DIVIDE
47     [Test]
48     public void Divide_Test()
49     {
50         Assert.AreEqual(5, calc.Divide(10, 2));
51     }
52
53     // DIVIDE BY ZERO
54     [Test]
55     public void Divide_ByZero_Test()
56     {
57         Assert.Throws<DivideByZeroException>(() => calc.Divide(10, 0));
58     }
59 }
60 }
```

## Test Execution Result:

The screenshot displays the Visual Studio IDE with the 'CalculatorLibrary' project selected. The Test Explorer on the left shows the results of a test run. The table below summarizes the test results:

| Test                       | Duration | Traits | Error Message |
|----------------------------|----------|--------|---------------|
| CalculatorLibrary.Test (7) | 12 ms    |        |               |
| CalculatorLibrary.Test (7) | 12 ms    |        |               |
| CalculatorTests (7)        | 12 ms    |        |               |

The Test Explorer also shows a summary of the test run: 7 Tests (7 Passed, 0 Failed, 0 Skipped) run in 189 ms. The Group Summary on the right indicates that all tests in the group passed, with a total duration of 12 ms.

The Output window at the bottom shows the build process. The build started at 02:07 AM and completed at 02:07 AM, taking 00.089 seconds. The build was successful, with 0 failed, 2 up-to-date, and 0 skipped tests.

```
Build started at 02:07 AM...
----- Build: 0 succeeded, 0 failed, 2 up-to-date, 0 skipped -----
----- Build completed at 02:07 AM and took 00.089 seconds -----
```